University of Technology Sydney
Faculty of Engineering and Information Technology

Subject 32547 UNIX Systems Programming, Autumn 2026

Assignment

Description

This assignment is an individual programming assignment using Python. It addresses Subject Learning Outcomes 2 and 3 as listed in the Subject Information document.

No limits apply to the number of lines of code of the program.

Assignments are **to be completed individually** (this might be checked with the use of anti-plagiarism tools such as Turnitin). **You should not receive help in the preparation of this assignment, nor ask anyone else to prepare it on your behalf, nor utilise generative AI tools such as ChatGPT, Claude, Gemini, Cursor or any others to generate the code**.

Marks

The assignment corresponds to 30% of the total marks.

Submission

The completed assignment is due by **11:59pm of Friday 15 May 2026**.

PLEASE PAY ATTENTION TO THE FOLLOWING SUBMISSION INSTRUCTIONS:

1. Your Python program has to be submitted in Canvas, following the "Assignments" menu and then the "Assignment" link by the due date. You can prepare your Python program anywhere you like, but probably the easiest option is to develop it in Ed STEM. You can submit your program multiple times if you like, and **we will mark your last submission**. Note that Canvas adds a small suffix to the filename ("-1", "-2" etc) for any submission after the first one, but it's perfectly fine, we'll remove the suffix ourselves before marking.
2. Please submit only your Python program, and nothing else (no data files).
3. Late submissions will be deducted one mark per day late; more than seven days late, the assignment will receive zero. Special considerations for a late submission must be arranged in advance with the Subject Coordinator.

Academic Standards

The assignments in this Subject should be your own original work. Any code taken from a textbook, journal, the Internet or any other source should be acknowledged. For referencing, please use the standard referencing conventions (http://www.lib.uts.edu.au/help/referencing).

Marking Scheme

Mark Range	
30	All requirements of the assignment are met. The submitted program can be executed by the lecturer "as is" and produces the requested output.
24-29	The program works correctly with any valid file and for all options, but its execution experiences some minor problems.
18-23	The program does not work as expected with one of the options.
0-17	This range goes from no submission at all (0 marks) to a submission which does not meet major requirements of the assignment. Examples: • the program does not work as expected with two or more options; • the program generates unhandled errors; • the program does not solve the assignment problem.

The assignments will be marked within two weeks from the submission deadline or as soon as possible.

Important notes:

- **Submission of this assignment is not compulsory to pass the subject**; do not feel that you have to submit it "at all costs" and perhaps be tempted to seek unauthorised assistance.
- There are no minimum requirements on the marks on this assignment to pass the subject.
- This assignment **must be your own work** and **you should not be helped to prepare it in any way**; your assignment may be tested with **anti-plagiarism software** that detects superficial changes such as changing variable names, swapping lines of code and the like.
- The subject coordinator **may ask you to describe your assignment in an online interview in Teams or Zoom** to finalise your mark.
- Understanding the assignment specifications is part of the assignment itself and no further instructions will be provided; on the other hand, whatever is not constrained you can implement it according to your own best judgment.

Title: Travel help with Python

In this assignment, you will write a Python program that can provide you with the travelling distance by car between some fictional cities.

These are the specifications for your Python program:

1. It must be named *travelhelper.py*

2. It must be invoked as:

   `python travelhelper.py` *`option argument_file`*

   In the command line above, *option* means one of the options described below, and *argument_file* means an **argument file**, which can have **any arbitrary name**, and is of the format described below.

   The program must immediately check that argument *argument_file* exists, is a file and is readable. If not, it must print an error message to **the standard output** and exit. The specifications for the *argument_file* and *option* arguments are given below.

3. File *argument_file* **can have any arbitrary name**. It must be a file of text with the following format:

   a. The file consists of an arbitrary number of lines (including, possibly, zero lines).

   b. Each line must contain three fields **separated by commas**.

   c. The three fields are: *origin, destination*, *distance*.

   d. The *origin* and *destination* fields are each a string of characters of arbitrary (yet reasonably limited) length. The characters only consist of lower- and upper-case letters (please note that they cannot include spaces). These two fields are meant to represent fictional cities names (origin and destination of a trip, respectively).

   e. The *distance* field is a string of digits representing a non-negative integer between 0 and 10000. This field is meant to represent the travelling distance in kilometers by car from the origin to the destination.

   Fundamental note: **your program is not expected to verify that file *argument_file* complies with the above specifications! It will only be tested with "compliant" files** (that is, files that follow the above specifications).

The following example should be regarded as the reference specification for the format of file *argument_file*:

```
Mornfall,Estbridge,142
Mornfall,Coldven,98
Estbridge,Mornfall,139
Dunhallow,Vireton,4
Mornfall,Dunhallow,211
Ashwick,Greyveil,520
Greyveil,Fenport,61
Mornfall,Crestlow,198
Crestlow,Fenport,174
Ashwick,Mornfall,834
```

4. Your program can be invoked with option: -a. In this case, your program must print every pair of origin and destination cities in the order in which they appear in the argument file, if any, in this format:

```
Origin and destination cities:
first origin >> first destination
..
last origin >> last destination
```

Example with the example *argument_file* given above (pay attention! Very important!):

Command line:

```
python travelhelper.py –a argument_file
```

Expected output:

```
Origin and destination cities:
Mornfall >> Estbridge
Mornfall >> Coldven
Estbridge >> Mornfall
Dunhallow >> Vireton
Mornfall >> Dunhallow
Ashwick >> Greyveil
Greyveil >> Fenport
Mornfall >> Crestlow
Crestlow >> Fenport
Ashwick >> Mornfall
```

In the case in which file *argument_file* is empty, your program must print:

```
No city distance information in this file
```

Please note that your program should only and exactly print the required strings and the required number of lines. Your program should not print any extra spaces or any other characters different from those required.

5. Your program can be invoked with option: -o *origin*. The *origin* argument has the same format as the *origin* field in the argument file. In this case, your program must search for this origin city in the argument file, and if it finds it, print the information of all its known destinations in this format:

```
Destinations from origin:
number. first destination (distance km)
...
number. last destination (distance km)
```

Example with the example *argument_file* given above (pay attention! Very important!):

Command line:

```
python travelhelper.py –o Ashwick argument_file
```

Expected output:

```
Destinations from Ashwick:
1. Greyveil (520 km)
2. Mornfall (834 km)
```

In the case in which city *origin* is not present in file *argument_file* (including the case where the file is empty), your program must print:

```
No known destinations from origin
```

Example with the *argument_file* given above:

Command line:

```
python travelhelper.py –o Bramwick argument_file
```

Expected output:

```
No known destinations from Bramwick
```

Please note that your program should only and exactly print the required strings and the required number of lines. Your program should not print any extra spaces or any other characters different from those required.

6. Your program can be invoked with option: -d *distance*. The *distance* argument has the same format as the *distance* field in the argument file. In this case, your program must search for every pair of origin and destination cities in the argument file whose distance is less than or equal to the *distance* argument, and if it finds any, print their information in the order in which they appear in the file in this format:

```
Cities within distance Km distance:
Distance: distance between first pair
first origin >> first destination
...
Distance: distance between last pair
last origin >> last destination
```

Example with the example *argument_file* given above (pay attention! Very important!):

Command line:

```
python travelhelper.py –d 139 argument_file
```

Expected output:

```
Cities within 139 Km distance:
Distance: 98
Mornfall >> Coldven
Distance: 139
Estbridge >> Mornfall
Distance: 4
Dunhallow >> Vireton
Distance: 61
Greyveil >> Fenport
```

In the case in which no origin and destination cities exist within the *distance* argument in file *argument_file* (including the case where the file is empty), your program must print:

```
No cities within distance Km
```

Example with the *argument_file* given above:

Command line:

```
python travelhelper.py –d 3 argument_file
```

Expected output:

```
No cities within 3 Km
```

Please note that your program should only and exactly print the required strings and the required number of lines. Your program should not print any extra spaces or any other characters different from those required.

7. Your program can be invoked with option: -v. In this case, it must only print your name, surname, student ID and date of completion of your assignment, in a format of your choice. Please note that argument *argument_file* is still required.

8. No options can be used simultaneously. This means that your program can only be invoked with one of the options at a time.

9. If your program is invoked with any other syntax than those specified above, it must print a message of your choice **<u>to the standard output</u>** and exit.

   Examples of incorrect syntax:

   `python travelhelper.py -Z argument_file` *(this option doesn't exist)*

   `python travelhelper.py –v` *(it misses the argument file)*

   `python travelhelper.py –o argument_file` *(it misses the origin argument)*

   `python travelhelper.py -d argument_file` *(it misses the distance argument)*